



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

КБ-3 «Разработка программных решений и системное программирование»

Отчет по практической работе №6

«Проект каталога: формы, роли и работа с данными»

по дисциплине

«Сборка, тестирование и верификация программного продукта»

Выполнил студент группы БСБО-09-23

Фёдоров Антон Сергеевич

Работа выполнена

«14» мая 2026 г.

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ЦЕЛЬ И ПЛАН РАБОТЫ	3
2. РОЛИ, ФОРМЫ И РАБОТА С ДАННЫМИ.....	4
3. ПАРТИИ ТОВАРА И РАСЧЕТ СУММЫ	7
4. ТЕСТ-ПЛАН И МАТРИЦА ТЕСТ-КЕЙСОВ.....	9
5. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ	13
6. ЧЕК-ЛИСТ ПО ТРЕБОВАНИЯМ	15
7. ИСХОДНЫЙ КОД	16
7.1. Модели.....	16
7.2. Формы.....	19
7.3. Представления.....	20
7.4. Маршрутизация.....	24
7.5. HTML-шаблоны	25
7.6. Тесты.....	29
ВЫВОДЫ	36

1. ЦЕЛЬ И ПЛАН РАБОТЫ

Цель: реализовать этап проекта каталога товаров на Django, связанный с работой с данными: добавить роли пользователей, формы управления товарами, механизм формирования партии товара на отправку, расчет суммы партии с учетом оптовых цен и дополнительной скидки, а также покрыть новый функционал автоматизированными тестами.

План:

1. Изучить требования к практической работе 6.
2. Использовать каталог аккумуляторов из предыдущих практических работ как основу.
3. Добавить роли: администратор, товаровед, менеджер продаж и гость.
4. Реализовать формы создания, редактирования и удаления товара для товароведа.
5. Добавить поля мелкого и крупного опта в модель товара.
6. Реализовать модели партии товара и позиции партии.
7. Добавить расчет суммы партии с учетом оптовых цен и дополнительной скидки.
8. Запретить менеджеру изменять чужие партии.
9. Обновить маршруты и HTML-шаблоны для новых операций.
10. Написать тесты форм, ролей, POST-запросов, маршрутов и расчетов.
11. Запустить тесты и coverage, затем оформить отчет.

2. РОЛИ, ФОРМЫ И РАБОТА С ДАННЫМИ

Тема каталога: аккумулятор. Проект Django находится в папке ПЗ6/catalog_project.

В ПЗ6 каталог из предыдущей работы расширен интерфейсом работы с данными. Список товаров, поиск, фильтрация, сортировка, карточка товара и страница About сохранены. Дополнительно добавлены операции управления товарами и партии на отправку.

В проекте используются четыре уровня доступа:

- гость не авторизован и может только просматривать каталог, карточки товаров и About;
- товаровед входит в группу Товароведы и может создавать, редактировать и удалять товары через интерфейс каталога;
- менеджер продаж входит в группу Менеджеры продаж и может создавать партии на отправку, добавлять товары в свои партии и менять дополнительную скидку;
- администратор определяется через `is_superuser` и имеет доступ ко всем операциям.

Роли проверяются во view-функциях через вспомогательные функции `is_product_specialist(user)`, `is_sales_manager(user)`, `role_required(...)` и `role_context(request)`. Декоратор `role_required` закрывает view от гостей и пользователей без нужной роли. Функция `role_context` передает в шаблоны признаки `can_manage_products` и `can_manage_shipments`, чтобы интерфейс показывал только доступные действия.

Для товара используется форма `BatteryForm`. Она содержит обычные характеристики аккумулятора и поля оптового ценообразования:

- `price` - цена за одну единицу;
- `small_wholesale_price` - цена мелкого опта;

- `small_wholesale_min_quantity` - количество для мелкого опта;
- `large_wholesale_price` - цена крупного опта;
- `large_wholesale_min_quantity` - количество для крупного опта.

Форма BatteryForm проверяет бизнес-правила: цена мелкого опта не должна быть выше цены за единицу, цена крупного опта не должна быть выше цены мелкого опта, а порог крупного опта должен быть больше порога мелкого опта.

Скриншоты интерфейса ПЗ6 сохранены в папке ПЗ6/screenshots/.

Рисунок 1. Каталог аккумуляторов с фильтрами, сортировкой и действиями по ролям

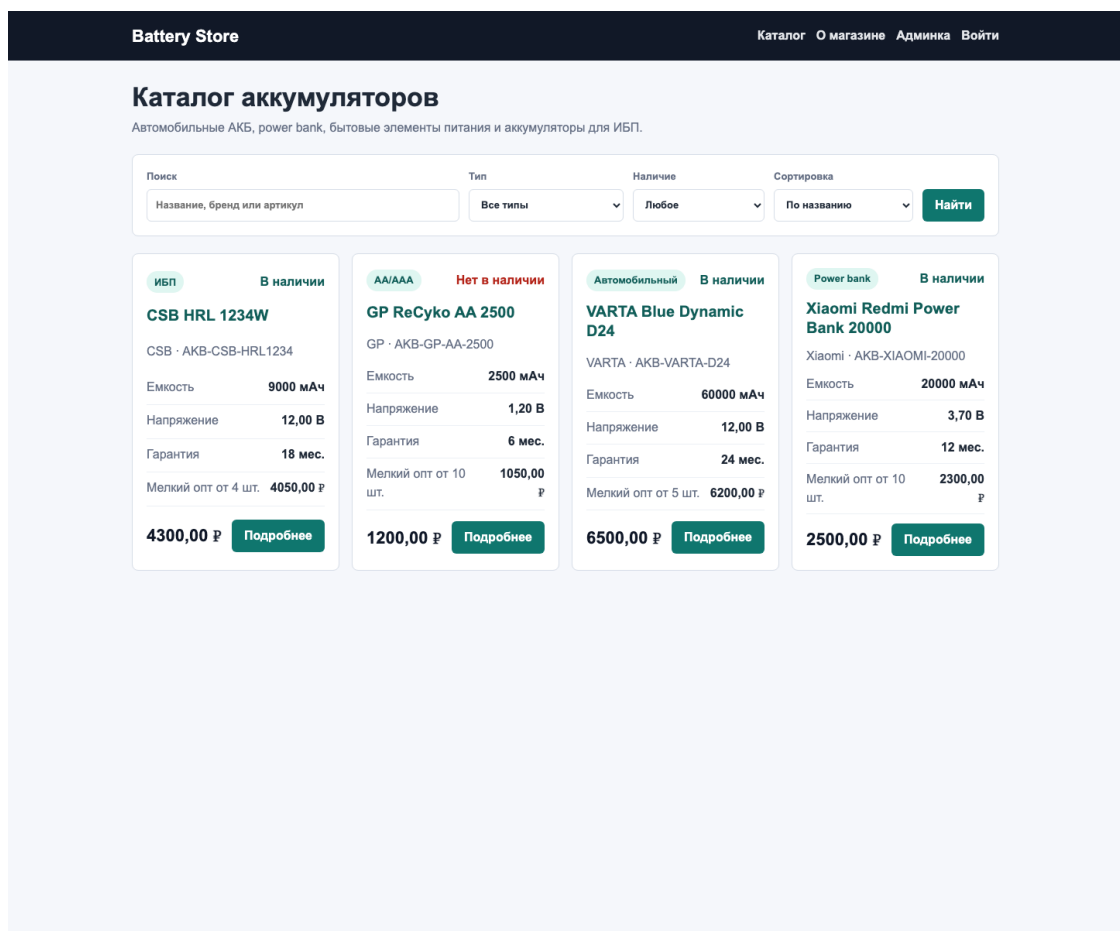


Рисунок 2. Карточка товара с оптовыми ценами и кнопками управления

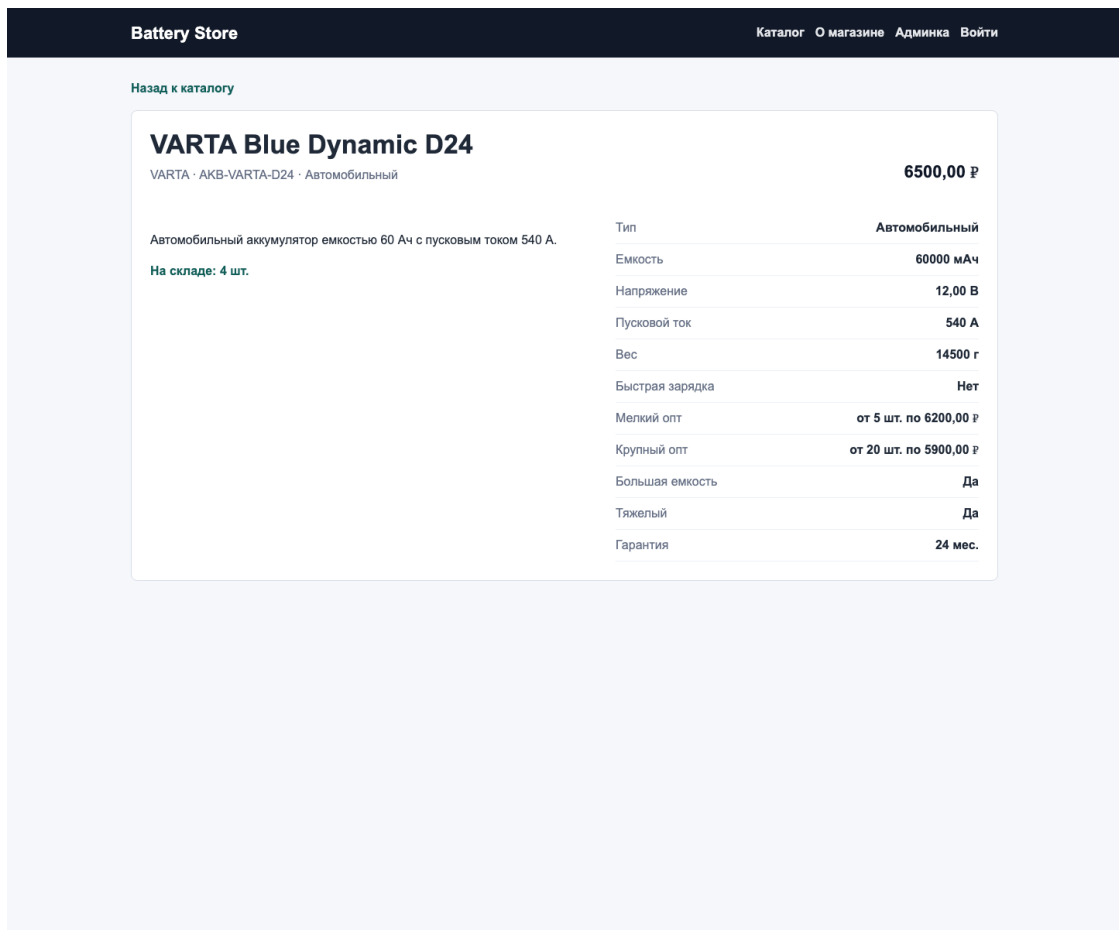


Рисунок 3. Страница входа пользователя

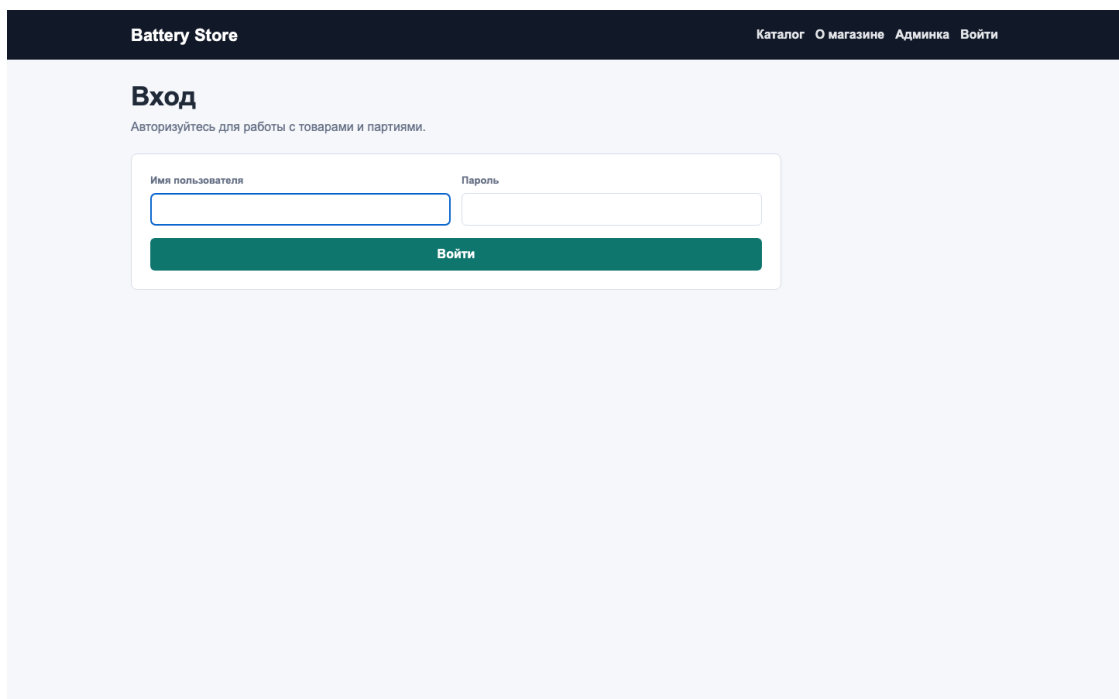


Рисунок 4. Форма создания и редактирования товара для товароведа

[Назад к каталогу](#)

Добавить аккумулятор

Данные товара доступны товароведу и администратору.

Артикул	Название
Бренд	Тип аккумулятора
	
Емкость, мАч	Напряжение, В
Пусковой ток, А	Вес, г
Цена за единицу, руб.	Цена мелкого опта, руб.
Количество для мелкого опта	Цена крупного опта, руб.
10	
Количество для крупного опта	Остаток на складе
50	0
Гарантия, мес.	
12	
Поддержка быстрой зарядки	☐
Описание	
Создать товар Отмена	

3. ПАРТИИ ТОВАРА И РАСЧЕТ СУММЫ

Модель Shipment играет роль корзины или партии товара на отправку. У партии есть владелец owner, название title и дополнительная скидка additional_discount_percent.

Позиции партии хранятся в модели ShipmentItem. Каждая позиция связана с товаром Battery и содержит количество. Для позиции рассчитываются unit_price и total_price. Цена единицы выбирается методом Battery.get_price_for_quantity(quantity):

- если количество достигло порога крупного опта, используется large_wholesale_price;
- иначе, если количество достигло порога мелкого опта, используется small_wholesale_price;
- иначе используется обычная цена price.

Для всей партии рассчитываются subtotal, discount_amount и total. Subtotal равен сумме всех позиций. Discount_amount рассчитывает дополнительную скидку от общей суммы. Total показывает итог после скидки.

Менеджер продаж видит и изменяет только свои партии. Если другой менеджер пытается открыть чужую партию, добавить в нее товар или удалить позицию, приложение возвращает 403 Forbidden. Администратор может открыть партии любого менеджера.

Локальные пользователи для ручной проверки:

- admin / admin12345 - администратор;
- tovaroved / worker12345 - товаровед;
- manager1 / manager12345 - менеджер продаж;
- manager2 / manager12345 - второй менеджер продаж.

Команды запуска проекта:

```
cd П36/catalog_project
python3 manage.py migrate
```

```
python3 manage.py loaddata battery_catalog
python3 manage.py runserver 127.0.0.1:8006
```

Рисунок 5. Список партий менеджера продаж

Battery Store				
Каталог Партии О магазине Админка manager1 Выйти				
Партии на отправку Менеджер видит и изменяет только свои партии. Администратор видит все. Новая партия				
Название	Менеджер	Позиций	Итого	
Партия для клиента	manager1	1	29450,00 ₽	Открыть

Рисунок 6. Карточка партии и расчет итоговой суммы

[Назад к партиям](#)

Партия для клиента

Менеджер: manager1

Название партии	Дополнительная скидка, %	Товар	Количество
Партия для клиента	5.00		
Обновить скидку		Добавить позицию	

Товар	Количество	Цена	Сумма	
VARTA Blue Dynamic D24	5	6200,00 ₽	31000,00 ₽	Удалить
Сумма до скидки			31000,00 ₽	
Дополнительная скидка			1550,00 ₽	
Итого			29450,00 ₽	

4. ТЕСТ-ПЛАН И МАТРИЦА ТЕСТ-КЕЙСОВ

Для практической работы подготовлен файл TEST_PLAN.md.
Идентификатор тест-плана: TP-AKB-2026-06.

Тест-план проверяет роли пользователей, формы изменения товаров, партии на отправку, POST-запросы и расчет итоговой суммы. В отличие от ПЗ5, здесь тестируются не только GET-страницы, но и операции создания, редактирования и удаления объектов.

Проверяемые модели: Battery, Shipment, ShipmentItem. Проверяемые формы: BatteryForm, ShipmentForm, ShipmentItemForm. Проверяемые view-функции: product_list, product_detail, product_create, product_update, product_delete, shipment_list, shipment_create, shipment_detail, shipment_add_item, shipment_delete_item и about.

Не тестируются Selenium/E2E-сценарии, внешний API, оплата, доставка, восстановление пароля, публичная регистрация, права внутри Django admin и конкурентное редактирование партии несколькими пользователями.

Критерий прохождения: пользователь с разрешенной ролью получает доступ к операции, пользователь без нужной роли получает 403 или redirect на login, после POST-запроса объект создается, изменяется или удаляется, менеджер видит только свои партии, а сумма партии совпадает с ручным расчетом.

Матрица тест-кейсов

- TC-01. Гость открывает каталог: GET / возвращает 200.
- TC-02. Гость открывает карточку товара: GET /product/<pk>/ возвращает 200.
- TC-03. Гость открывает About: GET /about/ возвращает 200.
- TC-04. Гость открывает форму добавления товара: redirect на login.
- TC-05. Гость открывает список партий: redirect на login.

- TC-06. Менеджер продаж открывает форму товара: 403.
- TC-07. Менеджер продаж открывает редактирование товара: 403.
- TC-08. Менеджер продаж открывает удаление товара: 403.
- TC-09. Товаровед создает товар через POST.
- TC-10. Товаровед редактирует описание и цены товара через POST.
- TC-11. Товаровед удаляет товар через POST.
- TC-12. Администратор открывает форму редактирования товара.
- TC-13. Менеджер создает собственную партию.
- TC-14. Менеджер добавляет товар в свою партию.
- TC-15. Менеджер обновляет скидку партии.
- TC-16. Менеджер видит только свои партии.
- TC-17. Другой менеджер не открывает чужую партию.
- TC-18. Другой менеджер не добавляет товар в чужую партию.
- TC-19. Другой менеджер не удаляет позицию из чужой партии.
- TC-20. Администратор открывает партию менеджера.
- TC-21. Shipment.total учитывает опт и общую скидку.
- TC-22. BatteryForm принимает валидные данные.
- TC-23. BatteryForm.save() сохраняет товар.
- TC-24. BatteryForm отклоняет дублирующийся артикул.
- TC-25. BatteryForm отклоняет цену мелкого опта выше розничной.
- TC-26. BatteryForm отклоняет цену крупного опта выше мелкого опта.
- TC-27. BatteryForm отклоняет неверный порог крупного опта.
- TC-28. URL CRUD товара строятся через reverse и resolve.
- TC-29. URL партий строятся через reverse и resolve.
- TC-30. Поиск и фильтрация каталога продолжают работать.
- TC-31. Сортировка каталога продолжает работать.

5. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Для проверки проекта использовался встроенный тестовый фреймворк Django на базе unittest.

Команда запуска тестов:

```
cd ПЗ6/catalog_project
python3 manage.py test catalog.tests --verbosity=2
```

Фактический результат:

```
Found 46 test(s).
Ran 46 tests in 3.561s
OK
```

Команды проверки покрытия:

```
python3 -m coverage run --source='.' manage.py test catalog.tests
python3 -m coverage report
```

Фактический результат покрытия:

Name	Stmts	Miss	Cover
-----	-----	-----	-----
catalog/forms.py	29	0	100%
catalog/models.py	107	5	95%
catalog/tests/test_forms.py	34	0	100%
catalog/tests/test_roles_and_shipments.py	115	0	100%
catalog/tests/test_routes.py	51	0	100%
catalog/views.py	135	11	92%
TOTAL	681	29	96%

В тестах проверяется:

- доступ гостя только к публичным страницам;
- запрет менеджеру продаж управлять товарами;
- создание, редактирование и удаление товара товароведом;
- доступ администратора к управлению товарами;
- создание партии менеджером продаж;
- добавление товара в собственную партию;
- отображение только собственных партий менеджера;
- запрет изменения чужой партии другим менеджером;
- доступ администратора к партиям менеджеров;
- расчет суммы партии с оптовыми ценами и дополнительной скидкой;
- валидация формы товара;
- reverse, resolve, GET-статусы и 404;
- поиск, фильтрация, сортировка и About из ПЗ5.

6. ЧЕК-ЛИСТ ПО ТРЕБОВАНИЯМ

- [x] Добавлено несколько ролей пользователей.
- [x] Администратор может выполнять все операции.
- [x] Товаровед может создавать, редактировать и удалять товары через интерфейс каталога.
- [x] Менеджер продаж может формировать партии товара на отправку.
- [x] Гость может только просматривать каталог, карточки товаров и About.
- [x] В товар добавлена цена за единицу.
- [x] Добавлена цена мелкого опта.
- [x] Добавлена цена крупного опта.
- [x] Добавлены количества, с которых начинается мелкий и крупный опт.
- [x] Реализована партия товара на отправку через Shipment.
- [x] Реализованы позиции партии через ShipmentItem.
- [x] Реализован расчет суммы партии с учетом оптовых цен.
- [x] Реализована дополнительная скидка на общую сумму партии.
- [x] Менеджер не может изменить партию другого менеджера.
- [x] Пользователей с одной ролью может быть несколько.
- [x] Новый функционал покрыт тестами.
- [x] Реализовано 46 автоматизированных тестов.
- [x] Все тесты проходят успешно.
- [x] Покрытие catalog/forms.py составляет 100%.
- [x] Общее покрытие проекта составляет 96%.
- [x] Сделаны скриншоты страниц.

7. ИСХОДНЫЙ КОД

7.1. Модели

Листинг 1. Файл catalog/models.py

```
from decimal import Decimal

from django.conf import settings
from django.core.validators import MaxValueValidator, MinValueValidator, RegexValidator
from django.db import models
from django.db.models import Q
from django.urls import reverse

class BatteryType(models.Model):
    name = models.CharField('Название типа', max_length=80, unique=True)
    slug = models.SlugField('Код для фильтра', max_length=90, unique=True)
    description = models.TextField('Описание', blank=True)

    class Meta:
        ordering = ('name',)
        verbose_name = 'Тип аккумулятора'
        verbose_name_plural = 'Типы аккумуляторов'

    def __str__(self):
        return self.name

class BatteryQuerySet(models.QuerySet):
    def by_type(self, battery_type):
        if isinstance(battery_type, BatteryType):
            return self.filter(battery_type=battery_type)
        return self.filter(battery_type__slug=battery_type)

    def in_stock(self):
        return self.filter(stock_quantity__gt=0)

    def high_capacity(self):
        return self.filter(capacity_mah__gte=20000)

    def heavy(self):
        return self.filter(weight_grams__gte=5000)

class Battery(models.Model):
    sku = models.CharField(
        'Артикул',
        max_length=32,
        unique=True,
        validators=[
            RegexValidator(
                regex=r'^[A-Z0-9-]+$ ',
                message='Артикул должен состоять из латинских заглавных букв, цифр и дефисов.',
            )
        ],
    )
    name = models.CharField('Название', max_length=120)
    brand = models.CharField('Бренд', max_length=80)
    battery_type = models.ForeignKey(
        BatteryType,
        on_delete=models.PROTECT,
        related_name='batteries',
        verbose_name='Тип аккумулятора',
    )
    capacity_mah = models.PositiveIntegerField(
        'Емкость, мАч',
        validators=[MinValueValidator(1), MaxValueValidator(500000)],
    )
    voltage_v = models.DecimalField(
        'Напряжение, В',
        max_digits=5,
        decimal_places=2,
        validators=[MinValueValidator(Decimal('0.01')), MaxValueValidator(Decimal('100.00'))],
    )
    starting_current_a = models.PositiveIntegerField(
        'Пусковой ток, А',
        null=True,
    )
```

```

        blank=True,
        validators=[MinValueValidator(1), MaxValueValidator(3000)],
    )
    weight_grams = models.PositiveIntegerField(
        'Вес, г',
        null=True,
        blank=True,
        validators=[MinValueValidator(1), MaxValueValidator(100000)],
    )
    price = models.DecimalField(
        'Цена за единицу, руб.',
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(Decimal('0.01'))],
    )
    small_wholesale_price = models.DecimalField(
        'Цена мелкого опта, руб.',
        max_digits=10,
        decimal_places=2,
        null=True,
        blank=True,
        validators=[MinValueValidator(Decimal('0.01'))],
    )
    small_wholesale_min_quantity = models.PositiveIntegerField(
        'Количество для мелкого опта',
        default=10,
        validators=[MinValueValidator(1), MaxValueValidator(100000)],
    )
    large_wholesale_price = models.DecimalField(
        'Цена крупного опта, руб.',
        max_digits=10,
        decimal_places=2,
        null=True,
        blank=True,
        validators=[MinValueValidator(Decimal('0.01'))],
    )
    large_wholesale_min_quantity = models.PositiveIntegerField(
        'Количество для крупного опта',
        default=50,
        validators=[MinValueValidator(1), MaxValueValidator(100000)],
    )
    stock_quantity = models.PositiveIntegerField(
        'Остаток на складе',
        default=0,
        validators=[MinValueValidator(0), MaxValueValidator(100000)],
    )
    warranty_months = models.PositiveSmallIntegerField(
        'Гарантия, мес.',
        default=12,
        validators=[MinValueValidator(0), MaxValueValidator(120)],
    )
    fast_charge = models.BooleanField('Поддержка быстрой зарядки', default=False)
    description = models.TextField('Описание', blank=True)
    created_at = models.DateTimeField('Создано', auto_now_add=True)
    updated_at = models.DateTimeField('Обновлено', auto_now=True)

    objects = BatteryQuerySet.as_manager()

    class Meta:
        ordering = ('name',)
        verbose_name = 'Аккумулятор'
        verbose_name_plural = 'Аккумуляторы'
        constraints = [
            models.CheckConstraint(condition=Q(price__gt=0), name='battery_price_positive'),
            models.CheckConstraint(condition=Q(capacity_mah__gt=0),
name='battery_capacity_positive'),
            models.CheckConstraint(condition=Q(voltage_v__gt=0),
name='battery_voltage_positive'),
            models.CheckConstraint(condition=Q(stock_quantity__gte=0),
name='battery_stock_non_negative'),
            models.CheckConstraint(
                condition=Q(weight_grams__isnull=True) | Q(weight_grams__gt=0),
                name='battery_weight_null_or_positive',
            ),
            models.CheckConstraint(
                condition=Q(starting_current_a__isnull=True) | Q(starting_current_a__gt=0),
                name='battery_starting_current_null_or_positive',
            ),
            models.CheckConstraint(
                condition=Q(warranty_months__gte=0) & Q(warranty_months__lte=120),
                name='battery_warranty_valid_range',
            ),
        ]

    @property
    def is_heavy(self):

```

```

        return self.weight_grams is not None and self.weight_grams >= 5000

@property
def is_high_capacity(self):
    return self.capacity_mah >= 20000

def get_price_for_quantity(self, quantity):
    if self.large_wholesale_price and quantity >= self.large_wholesale_min_quantity:
        return self.large_wholesale_price
    if self.small_wholesale_price and quantity >= self.small_wholesale_min_quantity:
        return self.small_wholesale_price
    return self.price

def get_absolute_url(self):
    return reverse('product_detail', args=[self.pk])

def __str__(self):
    return f'{self.brand} {self.name} ({self.sku})'

class Shipment(models.Model):
    owner = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='shipments',
        verbose_name='Менеджер',
    )
    title = models.CharField('Название партии', max_length=120, default='Партия на отправку')
    additional_discount_percent = models.DecimalField(
        'Дополнительная скидка, %',
        max_digits=5,
        decimal_places=2,
        default=Decimal('0.00'),
        validators=[MinValueValidator(Decimal('0.00')), MaxValueValidator(Decimal('100.00'))],
    )
    created_at = models.DateTimeField('Создано', auto_now_add=True)
    updated_at = models.DateTimeField('Обновлено', auto_now=True)

    class Meta:
        ordering = ('-updated_at',)
        verbose_name = 'Партия на отправку'
        verbose_name_plural = 'Партии на отправку'

    @property
    def subtotal(self):
        return sum((item.total_price for item in self.items.select_related('product'))),
        Decimal('0.00'))

    @property
    def discount_amount(self):
        return (self.subtotal * self.additional_discount_percent /
        Decimal('100')).quantize(Decimal('0.01'))

    @property
    def total(self):
        return self.subtotal - self.discount_amount

    def can_be_changed_by(self, user):
        return user.is_superuser or self.owner_id == user.id

    def __str__(self):
        return f'{self.title} ({self.owner})'

class ShipmentItem(models.Model):
    shipment = models.ForeignKey(
        Shipment,
        on_delete=models.CASCADE,
        related_name='items',
        verbose_name='Партия',
    )
    product = models.ForeignKey(
        Battery,
        on_delete=models.PROTECT,
        related_name='shipment_items',
        verbose_name='Товар',
    )
    quantity = models.PositiveIntegerField(
        'Количество',
        validators=[MinValueValidator(1), MaxValueValidator(100000)],
    )

```

```

class Meta:
    verbose_name = 'Позиция партии'
    verbose_name_plural = 'Позиции партии'

@property
def unit_price(self):
    return self.product.get_price_for_quantity(self.quantity)

@property
def total_price(self):
    return self.unit_price * self.quantity

def __str__(self):
    return f'{self.product} x {self.quantity}'

```

7.2. Формы

Листинг 2. Файл catalog/forms.py

```

from django import forms

from .models import Battery, Shipment, ShipmentItem

class BatteryForm(forms.ModelForm):
    class Meta:
        model = Battery
        fields = (
            'sku',
            'name',
            'brand',
            'battery_type',
            'capacity_mah',
            'voltage_v',
            'starting_current_a',
            'weight_grams',
            'price',
            'small_wholesale_price',
            'small_wholesale_min_quantity',
            'large_wholesale_price',
            'large_wholesale_min_quantity',
            'stock_quantity',
            'warranty_months',
            'fast_charge',
            'description',
        )
        widgets = {
            'description': forms.Textarea(attrs={'rows': 4}),
        }

    def clean(self):
        cleaned_data = super().clean()
        unit_price = cleaned_data.get('price')
        small_price = cleaned_data.get('small_wholesale_price')
        large_price = cleaned_data.get('large_wholesale_price')
        small_quantity = cleaned_data.get('small_wholesale_min_quantity')
        large_quantity = cleaned_data.get('large_wholesale_min_quantity')

        if unit_price and small_price and small_price > unit_price:
            self.add_error('small_wholesale_price', 'Цена мелкого опта не должна быть выше цены за единицу.')
        if small_price and large_price and large_price > small_price:
            self.add_error('large_wholesale_price', 'Цена крупного опта не должна быть выше цены мелкого опта.')
        if small_quantity and large_quantity and large_quantity <= small_quantity:
            self.add_error('large_wholesale_min_quantity', 'Количество крупного опта должно быть больше мелкого опта.')

        return cleaned_data

class ShipmentForm(forms.ModelForm):
    class Meta:
        model = Shipment
        fields = ('title', 'additional_discount_percent')

class ShipmentItemForm(forms.ModelForm):

```

```
class Meta:
    model = ShipmentItem
    fields = ('product', 'quantity')
```

7.3. Представления

Листинг 3. Файл catalog/views.py

```
from functools import wraps

from django.contrib.auth.views import redirect_to_login
from django.db.models import Q
from django.http import HttpResponseForbidden, HttpResponseNotAllowed
from django.shortcuts import get_object_or_404, redirect, render

from .forms import BatteryForm, ShipmentForm, ShipmentItemForm
from .models import Battery, BatteryType, Shipment, ShipmentItem

PRODUCT_SPECIALIST_GROUP = 'Товароведы'
SALES_MANAGER_GROUP = 'Менеджеры продаж'

SORT_OPTIONS = {
    'name': {
        'label': 'По названию',
        'order_by': ('name',),
    },
    'price': {
        'label': 'По цене',
        'order_by': ('price', 'name'),
    },
    'weight': {
        'label': 'По весу',
        'order_by': ('weight_grams', 'name'),
    },
    'type': {
        'label': 'По типу',
        'order_by': ('battery_type__name', 'name'),
    },
}
DEFAULT_SORT = 'name'

def is_product_specialist(user):
    return user.is_authenticated and (
        user.is_superuser or user.groups.filter(name=PRODUCT_SPECIALIST_GROUP).exists()
    )

def is_sales_manager(user):
    return user.is_authenticated and (
        user.is_superuser or user.groups.filter(name=SALES_MANAGER_GROUP).exists()
    )

def role_required(check):
    def decorator(view_func):
        @wraps(view_func)
        def wrapper(request, *args, **kwargs):
            if not request.user.is_authenticated:
                return redirect_to_login(request.get_full_path())
            if not check(request.user):
                return HttpResponseForbidden('Недостаточно прав для выполнения операции.')
            return view_func(request, *args, **kwargs)

        return wrapper

    return decorator

def role_context(request):
    return {
        'can_manage_products': is_product_specialist(request.user),
        'can_manage_shipments': is_sales_manager(request.user),
    }
```

```

def product_list(request):
    products = Battery.objects.select_related('battery_type')
    query = request.GET.get('q', '').strip()
    selected_type = request.GET.get('type', '').strip()
    selected_availability = request.GET.get('availability', '').strip()
    selected_sort = request.GET.get('sort', DEFAULT_SORT).strip()

    if query:
        products = products.filter(
            Q(name__icontains=query)
            | Q(brand__icontains=query)
            | Q(sku__icontains=query)
            | Q(description__icontains=query)
        )

    if selected_type:
        products = products.by_type(selected_type)

    if selected_availability == 'in_stock':
        products = products.in_stock()
    elif selected_availability == 'out_of_stock':
        products = products.filter(stock_quantity=0)

    if selected_sort not in SORT_OPTIONS:
        selected_sort = DEFAULT_SORT
    products = products.order_by(*SORT_OPTIONS[selected_sort]['order_by'])

    context = role_context(request) | {
        'products': products,
        'battery_types': BatteryType.objects.all(),
        'query': query,
        'selected_type': selected_type,
        'selected_availability': selected_availability,
        'selected_sort': selected_sort,
        'sort_options': SORT_OPTIONS,
    }
    return render(request, 'catalog/product_list.html', context)

def product_detail(request, pk):
    product = get_object_or_404(Battery.objects.select_related('battery_type'), pk=pk)
    return render(request, 'catalog/product_detail.html', role_context(request) | {'product': product})

@role_required(is_product_specialist)
def product_create(request):
    if request.method == 'POST':
        form = BatteryForm(request.POST)
        if form.is_valid():
            product = form.save()
            return redirect(product)
    else:
        form = BatteryForm()

    context = role_context(request) | {
        'form': form,
        'title': 'Добавить аккумулятор',
        'submit_label': 'Создать товар',
    }
    return render(request, 'catalog/product_form.html', context)

@role_required(is_product_specialist)
def product_update(request, pk):
    product = get_object_or_404(Battery, pk=pk)
    if request.method == 'POST':
        form = BatteryForm(request.POST, instance=product)
        if form.is_valid():
            product = form.save()
            return redirect(product)
    else:
        form = BatteryForm(instance=product)

    context = role_context(request) | {
        'form': form,
        'product': product,
        'title': 'Редактировать аккумулятор',
        'submit_label': 'Сохранить изменения',
    }

```

```

        return render(request, 'catalog/product_form.html', context)

@role_required(is_product_specialist)
def product_delete(request, pk):
    product = get_object_or_404(Battery, pk=pk)
    if request.method == 'POST':
        product.delete()
        return redirect('product_list')

    context = role_context(request) | {'product': product}
    return render(request, 'catalog/product_confirm_delete.html', context)

def get_allowed_shipment_or_404(request, pk):
    queryset = Shipment.objects.select_related('owner').prefetch_related('items__product')
    shipment = get_object_or_404(queryset, pk=pk)
    if not shipment.can_be_changed_by(request.user):
        return None
    return shipment

@role_required(is_sales_manager)
def shipment_list(request):
    shipments = Shipment.objects.select_related('owner').prefetch_related('items__product')
    if not request.user.is_superuser:
        shipments = shipments.filter(owner=request.user)

    return render(request, 'catalog/shipment_list.html', role_context(request) | {'shipments':
shipments})

@role_required(is_sales_manager)
def shipment_create(request):
    if request.method != 'POST':
        return HttpResponseRedirect(['POST'])

    shipment = Shipment.objects.create(owner=request.user)
    return redirect('shipment_detail', pk=shipment.pk)

@role_required(is_sales_manager)
def shipment_detail(request, pk):
    shipment = get_allowed_shipment_or_404(request, pk)
    if shipment is None:
        return HttpResponseRedirect('Эта партия принадлежит другому менеджеру.')

    if request.method == 'POST':
        form = ShipmentForm(request.POST, instance=shipment)
        if form.is_valid():
            form.save()
            return redirect('shipment_detail', pk=shipment.pk)
    else:
        form = ShipmentForm(instance=shipment)

    context = role_context(request) | {
        'shipment': shipment,
        'form': form,
        'item_form': ShipmentItemForm(),
    }
    return render(request, 'catalog/shipment_detail.html', context)

@role_required(is_sales_manager)
def shipment_add_item(request, pk):
    if request.method != 'POST':
        return HttpResponseRedirect(['POST'])

    shipment = get_allowed_shipment_or_404(request, pk)
    if shipment is None:
        return HttpResponseRedirect('Эта партия принадлежит другому менеджеру.')

    form = ShipmentItemForm(request.POST)
    if form.is_valid():
        item = form.save(commit=False)
        item.shipment = shipment
        item.save()

    return redirect('shipment_detail', pk=shipment.pk)

```

```

@role_required(is_sales_manager)
def shipment_delete_item(request, shipment_pk, item_pk):
    if request.method != 'POST':
        return HttpResponseRedirect(['POST'])

    shipment = get_allowed_shipment_or_404(request, shipment_pk)
    if shipment is None:
        return HttpResponseRedirect('Эта партия принадлежит другому менеджеру.')

    item = get_object_or_404(ShipmentItem, pk=item_pk, shipment=shipment)
    item.delete()
    return redirect('shipment_detail', pk=shipment.pk)

def about(request):
    return render(request, 'catalog/about.html', role_context(request))

```

7.4. Маршрутизация

Листинг 4. Файл catalog_project/urls.py

```

"""
URL configuration for catalog_project project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/6.0/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('accounts/', include('django.contrib.auth.urls')),
    path('', include('catalog.urls')),
]

```

Листинг 5. Файл catalog/urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.product_list, name='product_list'),
    path('products/add/', views.product_create, name='product_create'),
    path('product/<int:pk>/', views.product_detail, name='product_detail'),
    path('product/<int:pk>/edit/', views.product_update, name='product_update'),
    path('product/<int:pk>/delete/', views.product_delete, name='product_delete'),
    path('shipments/', views.shipment_list, name='shipment_list'),
    path('shipments/new/', views.shipment_create, name='shipment_create'),
    path('shipments/<int:pk>/', views.shipment_detail, name='shipment_detail'),
    path('shipments/<int:pk>/add-item/', views.shipment_add_item, name='shipment_add_item'),
    path(
        'shipments/<int:shipment_pk>/items/<int:item_pk>/delete/',
        views.shipment_delete_item,
        name='shipment_delete_item',
    ),
    path('about/', views.about, name='about'),
]

```

7.5. HTML-шаблоны

В ПЗ6 добавлены шаблоны форм товара, подтверждения удаления, списка партий, карточки партии и страницы входа. Ниже приведены ключевые шаблоны, связанные с новым функционалом.

Листинг 6. Файл catalog/templates/catalog/product_form.html

```
{% extends 'catalog/base.html' %}

{% block title %}{{ title }}{% endblock %}

{% block content %}
<a class="back-link" href="{% url 'product_list' %}">Назад к каталогу</a>

<section class="page-header">
  <div>
    <h1>{{ title }}</h1>
    <p>Данные товара доступны товароведу и администратору.</p>
  </div>
</section>

<form class="form-panel" method="post">
  {% csrf_token %}
  {{ form.non_field_errors }}
  <div class="form-grid">
    {% for field in form %}
      <label{% if field.name == 'description' or field.name == 'fast_charge' %}
class="wide"{% endif %}>
        {{ field.label }}
        {{ field }}
        {{ field.errors }}
      </label>
    {% endfor %}
  </div>
  <div class="actions">
    <button class="button" type="submit">{{ submit_label }}</button>
    {% if product %}
      <a class="button button--muted" href="{{ product.get_absolute_url }}">Отмена</a>
    {% else %}
      <a class="button button--muted" href="{% url 'product_list' %}">Отмена</a>
    {% endif %}
  </div>
</form>
{% endblock %}
```

Листинг 7. Файл catalog/templates/catalog/product_confirm_delete.html

```
{% extends 'catalog/base.html' %}

{% block title %}Удалить {{ product.name }}{% endblock %}

{% block content %}
<a class="back-link" href="{{ product.get_absolute_url }}">Назад к товару</a>

<section class="detail-panel">
  <div>
    <h1>Удалить товар</h1>
    <p>Товар «{{ product.name }}» будет удален из каталога.</p>
  </div>
  <form class="actions" method="post">
    {% csrf_token %}
    <button class="button button--danger" type="submit">Удалить</button>
    <a class="button button--muted" href="{{ product.get_absolute_url }}">Отмена</a>
  </form>
</section>
{% endblock %}
```

Листинг 8. Файл catalog/templates/catalog/shipment_list.html

```
{% extends 'catalog/base.html' %}
```

```

{% block title %}Партии на отправку{% endblock %}

{% block content %}
<section class="page-header">
  <div>
    <h1>Партии на отправку</h1>
    <p>Менеджер видит и изменяет только свои партии. Администратор видит все.</p>
  </div>
  <form method="post" action="{% url 'shipment_create' %}">
    {% csrf_token %}
    <button class="button" type="submit">Новая партия</button>
  </form>
</section>

{% if shipments %}
  <table class="data-table">
    <thead>
      <tr>
        <th>Название</th>
        <th>Менеджер</th>
        <th>Позиций</th>
        <th>Итого</th>
        <th></th>
      </tr>
    </thead>
    <tbody>
      {% for shipment in shipments %}
        <tr>
          <td>{{ shipment.title }}</td>
          <td>{{ shipment.owner.username }}</td>
          <td>{{ shipment.items.count }}</td>
          <td>{{ shipment.total }} P</td>
          <td><a href="{% url 'shipment_detail' shipment.pk %}">Открыть</a></td>
        </tr>
      {% endfor %}
    </tbody>
  </table>
{% else %}
  <section class="empty-state">
    <p>Партии пока не сформированы.</p>
  </section>
{% endif %}
{% endblock %}

```

Листинг 9. Файл catalog/templates/catalog/shipment_detail.html

```

{% extends 'catalog/base.html' %}

{% block title %}{{ shipment.title }}{% endblock %}

{% block content %}
<a class="back-link" href="{% url 'shipment_list' %}">Назад к партиям</a>

<section class="page-header">
  <div>
    <h1>{{ shipment.title }}</h1>
    <p>Менеджер: {{ shipment.owner.username }}</p>
  </div>
</section>

<section class="tool-grid">
  <form class="form-panel" method="post">
    {% csrf_token %}
    {{ form.non_field_errors }}
    <div class="form-grid">
      {% for field in form %}
        <label>
          {{ field.label }}
          {{ field }}
          {{ field.errors }}
        </label>
      {% endfor %}
    </div>
    <button class="button" type="submit">Обновить скидку</button>
  </form>

  <form class="form-panel" method="post" action="{% url 'shipment_add_item' shipment.pk %}">
    {% csrf_token %}
    {{ item_form.non_field_errors }}
    <div class="form-grid">

```

```

        {% for field in item_form %}
            <label>
                {{ field.label }}
                {{ field }}
                {{ field.errors }}
            </label>
        {% endfor %}
    </div>
    <button class="button" type="submit">Добавить позицию</button>
</form>
</section>

{% if shipment.items.all %}
    <table class="data-table">
        <thead>
            <tr>
                <th>Товар</th>
                <th>Количество</th>
                <th>Цена</th>
                <th>Сумма</th>
            </tr>
        </thead>
        <tbody>
            {% for item in shipment.items.all %}
                <tr>
                    <td>{{ item.product.name }}</td>
                    <td>{{ item.quantity }}</td>
                    <td>{{ item.unit_price }} Р</td>
                    <td>{{ item.total_price }} Р</td>
                </tr>
                <td colspan="4">
                    <form method="post" action="{% url 'shipment_delete_item' shipment.pk
item.pk %}">
                        {% csrf_token %}
                        <button class="button button--danger" type="submit">Удалить</button>
                    </form>
                </td>
            </tr>
        {% endfor %}
    </tbody>
    <tfoot>
        <tr>
            <th colspan="3">Сумма до скидки</th>
            <th colspan="2">{{ shipment.subtotal }} Р</th>
        </tr>
        <tr>
            <th colspan="3">Дополнительная скидка</th>
            <th colspan="2">{{ shipment.discount_amount }} Р</th>
        </tr>
        <tr>
            <th colspan="3">Итого</th>
            <th colspan="2">{{ shipment.total }} Р</th>
        </tr>
    </tfoot>
    </table>
{% else %}
    <section class="empty-state">
        <p>В партии пока нет товаров.</p>
    </section>
{% endif %}
{% endblock %}

```

Листинг 10. Файл catalog/templates/registration/login.html

```

{% extends 'catalog/base.html' %}

{% block title %}Вход{% endblock %}

{% block content %}
<section class="page-header">
    <div>
        <h1>Вход</h1>
        <p>Авторизуйтесь для работы с товарами и партиями.</p>
    </div>
</section>

<form class="form-panel" method="post">
    {% csrf_token %}
    {{ form.non_field_errors }}
    <div class="form-grid">
        {% for field in form %}
            <label>

```

```

        {{ field.label }}
        {{ field }}
        {{ field.errors }}
    </label>
{% endfor %}
</div>
<button class="button" type="submit">Войти</button>
</form>
{% endblock %}

```

7.6. Тесты

Листинг 11. Файл catalog/tests/helpers.py

```

from django.contrib.auth import get_user_model
from django.contrib.auth.models import Group

from catalog.models import Battery, BatteryType
from catalog.views import PRODUCT_SPECIALIST_GROUP, SALES_MANAGER_GROUP

PASSWORD = 'test-password-123'

def create_user(username, group_name=None, is_superuser=False):
    user_model = get_user_model()
    if is_superuser:
        return user_model.objects.create_superuser(
            username=username,
            email=f'{username}@example.com',
            password=PASSWORD,
        )

    user = user_model.objects.create_user(
        username=username,
        email=f'{username}@example.com',
        password=PASSWORD,
    )
    if group_name:
        group, _ = Group.objects.get_or_create(name=group_name)
        user.groups.add(group)
    return user

def create_product_specialist(username='specialist'):
    return create_user(username, PRODUCT_SPECIALIST_GROUP)

def create_sales_manager(username='manager'):
    return create_user(username, SALES_MANAGER_GROUP)

def create_battery_type(name='Автомобильный', slug='car'):
    return BatteryType.objects.create(
        name=name,
        slug=slug,
        description='Тестовый тип аккумуляторов.',
    )

def create_battery(battery_type=None, **overrides):
    if battery_type is None:
        battery_type = create_battery_type()

    data = {
        'sku': 'AKB-TEST-001',
        'name': 'Test Battery 60Ah',
        'brand': 'TestBrand',
        'battery_type': battery_type,
        'capacity_mah': 60000,
        'voltage_v': '12.00',
        'starting_current_a': 540,
        'weight_grams': 14500,
        'price': '6500.00',
        'small_wholesale_price': '6200.00',
        'small_wholesale_min_quantity': 5,
        'large_wholesale_price': '5900.00',
    }

```

```

        'large_wholesale_min_quantity': 20,
        'stock_quantity': 4,
        'warranty_months': 24,
        'description': 'Тестовый аккумулятор.',
    }
    data.update(overrides)
    return Battery.objects.create(**data)

def battery_form_data(battery_type, **overrides):
    data = {
        'sku': 'AKB-FORM-001',
        'name': 'Form Battery',
        'brand': 'FormBrand',
        'battery_type': str(battery_type.pk),
        'capacity_mah': '60000',
        'voltage_v': '12.00',
        'starting_current_a': '540',
        'weight_grams': '14500',
        'price': '6500.00',
        'small_wholesale_price': '6200.00',
        'small_wholesale_min_quantity': '5',
        'large_wholesale_price': '5900.00',
        'large_wholesale_min_quantity': '20',
        'stock_quantity': '4',
        'warranty_months': '24',
        'fast_charge': '',
        'description': 'Описание из формы.',
    }
    data.update(overrides)
    return data

```

Листинг 12. Файл catalog/tests/test_forms.py

```

from django.test import TestCase

from catalog.forms import BatteryForm
from catalog.models import Battery
from catalog.tests.helpers import battery_form_data, create_battery, create_battery_type

class BatteryFormTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.battery_type = create_battery_type()

    def test_battery_form_accepts_valid_product_data(self):
        form = BatteryForm(data=battery_form_data(self.battery_type))

        self.assertTrue(form.is_valid(), form.errors)

    def test_battery_form_creates_product(self):
        form = BatteryForm(data=battery_form_data(self.battery_type))

        self.assertTrue(form.is_valid(), form.errors)
        product = form.save()

        self.assertEqual(product.sku, 'AKB-FORM-001')
        self.assertEqual(Battery.objects.count(), 1)

    def test_battery_form_rejects_duplicate_sku(self):
        create_battery(self.battery_type, sku='AKB-FORM-001')

        form = BatteryForm(data=battery_form_data(self.battery_type))

        self.assertFalse(form.is_valid())
        self.assertIn('sku', form.errors)

    def test_battery_form_rejects_small_wholesale_price_above_unit_price(self):
        form = BatteryForm(
            data=battery_form_data(
                self.battery_type,
                small_wholesale_price='7000.00',
            )
        )

        self.assertFalse(form.is_valid())
        self.assertIn('small_wholesale_price', form.errors)

```

```

def test_battery_form_rejects_large_wholesale_price_above_small_wholesale_price(self):
    form = BatteryForm(
        data=battery_form_data(
            self.battery_type,
            large_wholesale_price='6300.00',
        )
    )

    self.assertFalse(form.is_valid())
    self.assertIn('large_wholesale_price', form.errors)

def test_battery_form_rejects_large_wholesale_quantity_not_greater_than_small(self):
    form = BatteryForm(
        data=battery_form_data(
            self.battery_type,
            large_wholesale_min_quantity='5',
        )
    )

    self.assertFalse(form.is_valid())
    self.assertIn('large_wholesale_min_quantity', form.errors)

```

Листинг 13. Файл catalog/tests/test_roles_and_shipments.py

```

from decimal import Decimal

from django.test import TestCase
from django.urls import reverse

from catalog.models import Battery, Shipment, ShipmentItem
from catalog.tests.helpers import (
    PASSWORD,
    battery_form_data,
    create_battery,
    create_battery_type,
    create_product_specialist,
    create_sales_manager,
    create_user,
)

class RoleAccessTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.battery_type = create_battery_type()
        cls.product = create_battery(cls.battery_type, sku='AKB-ROLE-001')
        cls.specialist = create_product_specialist()
        cls.manager = create_sales_manager()
        cls.admin = create_user('admin', is_superuser=True)

    def test_guest_can_only_view_catalog_pages(self):
        list_response = self.client.get(reverse('product_list'))
        detail_response = self.client.get(reverse('product_detail', args=[self.product.pk]))
        create_response = self.client.get(reverse('product_create'))
        shipment_response = self.client.get(reverse('shipment_list'))

        self.assertEqual(list_response.status_code, 200)
        self.assertEqual(detail_response.status_code, 200)
        self.assertEqual(create_response.status_code, 302)
        self.assertIn('/accounts/login/', create_response.url)
        self.assertEqual(shipment_response.status_code, 302)
        self.assertIn('/accounts/login/', shipment_response.url)

    def test_sales_manager_cannot_manage_catalog_products(self):
        self.client.login(username=self.manager.username, password=PASSWORD)

        create_response = self.client.get(reverse('product_create'))
        update_response = self.client.get(reverse('product_update', args=[self.product.pk]))
        delete_response = self.client.get(reverse('product_delete', args=[self.product.pk]))

        self.assertEqual(create_response.status_code, 403)
        self.assertEqual(update_response.status_code, 403)
        self.assertEqual(delete_response.status_code, 403)

    def test_product_specialist_can_create_product(self):
        self.client.login(username=self.specialist.username, password=PASSWORD)

        response = self.client.post(

```

```

        reverse('product_create'),
        battery_form_data(self.battery_type, sku='AKB-CREATE-001'),
    )

    self.assertEqual(response.status_code, 302)
    self.assertTrue(Battery.objects.filter(sku='AKB-CREATE-001').exists())

def test_product_specialist_can_update_product_description_and_prices(self):
    self.client.login(username=self.specialist.username, password=PASSWORD)

    response = self.client.post(
        reverse('product_update', args=[self.product.pk]),
        battery_form_data(
            self.battery_type,
            sku=self.product.sku,
            description='Обновленное описание товароведа.',
            price='6400.00',
            small_wholesale_price='6100.00',
            large_wholesale_price='5800.00',
        ),
    )

    self.assertEqual(response.status_code, 302)
    self.product.refresh_from_db()
    self.assertEqual(self.product.description, 'Обновленное описание товароведа.')
    self.assertEqual(self.product.price, Decimal('6400.00'))

def test_product_specialist_can_delete_product(self):
    product = create_battery(self.battery_type, sku='AKB-DELETE-001')
    self.client.login(username=self.specialist.username, password=PASSWORD)

    response = self.client.post(reverse('product_delete', args=[product.pk]))

    self.assertEqual(response.status_code, 302)
    self.assertFalse(Battery.objects.filter(pk=product.pk).exists())

def test_admin_can_manage_products(self):
    self.client.login(username=self.admin.username, password=PASSWORD)

    response = self.client.get(reverse('product_update', args=[self.product.pk]))

    self.assertEqual(response.status_code, 200)

class ShipmentTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.battery_type = create_battery_type()
        cls.product = create_battery(cls.battery_type, sku='AKB-SHIP-001')
        cls.manager = create_sales_manager('manager-one')
        cls.other_manager = create_sales_manager('manager-two')
        cls.admin = create_user('admin', is_superuser=True)

    def test_sales_manager_can_create_own_shipment(self):
        self.client.login(username=self.manager.username, password=PASSWORD)

        response = self.client.post(reverse('shipment_create'))

        self.assertEqual(response.status_code, 302)
        shipment = Shipment.objects.get()
        self.assertEqual(shipment.owner, self.manager)

    def test_sales_manager_can_add_product_to_own_shipment(self):
        shipment = Shipment.objects.create(owner=self.manager)
        self.client.login(username=self.manager.username, password=PASSWORD)

        response = self.client.post(
            reverse('shipment_add_item', args=[shipment.pk]),
            {'product': self.product.pk, 'quantity': 5},
        )

        self.assertEqual(response.status_code, 302)
        item = ShipmentItem.objects.get(shipment=shipment)
        self.assertEqual(item.quantity, 5)
        self.assertEqual(item.unit_price, Decimal('6200.00'))

    def test_manager_sees_only_own_shipments(self):
        own_shipment = Shipment.objects.create(owner=self.manager, title='Своя партия')
        Shipment.objects.create(owner=self.other_manager, title='Чужая партия')
        self.client.login(username=self.manager.username, password=PASSWORD)

```

```

response = self.client.get(reverse('shipment_list'))

self.assertContains(response, own_shipment.title)
self.assertNotContains(response, 'Чужая партия')

def test_one_manager_cannot_change_another_manager_shipment(self):
    shipment = Shipment.objects.create(owner=self.manager)
    item = ShipmentItem.objects.create(shipment=shipment, product=self.product, quantity=3)
    self.client.login(username=self.other_manager.username, password=PASSWORD)

    detail_response = self.client.get(reverse('shipment_detail', args=[shipment.pk]))
    add_response = self.client.post(
        reverse('shipment_add_item', args=[shipment.pk]),
        {'product': self.product.pk, 'quantity': 2},
    )
    delete_response = self.client.post(
        reverse('shipment_delete_item', args=[shipment.pk, item.pk]),
    )

    self.assertEqual(detail_response.status_code, 403)
    self.assertEqual(add_response.status_code, 403)
    self.assertEqual(delete_response.status_code, 403)
    self.assertEqual(shipment.items.count(), 1)

def test_admin_can_open_any_manager_shipment(self):
    shipment = Shipment.objects.create(owner=self.manager)
    self.client.login(username=self.admin.username, password=PASSWORD)

    response = self.client.get(reverse('shipment_detail', args=[shipment.pk]))

    self.assertEqual(response.status_code, 200)

def test_shipment_total_uses_wholesale_prices_and_additional_discount(self):
    shipment = Shipment.objects.create(
        owner=self.manager,
        additional_discount_percent=Decimal('10.00'),
    )
    ShipmentItem.objects.create(shipment=shipment, product=self.product, quantity=3)
    ShipmentItem.objects.create(shipment=shipment, product=self.product, quantity=5)
    ShipmentItem.objects.create(shipment=shipment, product=self.product, quantity=20)

    self.assertEqual(shipment.subtotal, Decimal('168500.00'))
    self.assertEqual(shipment.discount_amount, Decimal('16850.00'))
    self.assertEqual(shipment.total, Decimal('151650.00'))

def test_sales_manager_can_update_additional_discount(self):
    shipment = Shipment.objects.create(owner=self.manager)
    self.client.login(username=self.manager.username, password=PASSWORD)

    response = self.client.post(
        reverse('shipment_detail', args=[shipment.pk]),
        {'title': 'Партия для клиента', 'additional_discount_percent': '7.50'},
    )

    self.assertEqual(response.status_code, 302)
    shipment.refresh_from_db()
    self.assertEqual(shipment.title, 'Партия для клиента')
    self.assertEqual(shipment.additional_discount_percent, Decimal('7.50'))

```

Листинг 14. Файл catalog/tests/test_routes.py

```

from django.test import TestCase
from django.urls import resolve, reverse

from catalog import views
from catalog.models import Battery, BatteryType, Shipment, ShipmentItem
from catalog.tests.helpers import create_sales_manager

class CatalogRouteTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        battery_type = BatteryType.objects.create(name='Автомобильный', slug='car')
        cls.product = Battery.objects.create(
            sku='AKB-VARTA-D24',
            name='VARTA Blue Dynamic D24',
            brand='VARTA',

```

```

        battery_type=battery_type,
        capacity_mah=60000,
        voltage_v='12.00',
        starting_current_a=540,
        weight_grams=14500,
        price='6500.00',
        stock_quantity=4,
        warranty_months=24,
        description='Автомобильный аккумулятор емкостью 60 Ач с пусковым током 540 А.',
    )
    cls.manager = create_sales_manager()
    cls.shipment = Shipment.objects.create(owner=cls.manager)
    cls.shipment_item = ShipmentItem.objects.create(
        shipment=cls.shipment,
        product=cls.product,
        quantity=2,
    )

def test_product_list_url_resolves_to_product_list_view(self):
    match = resolve(reverse('product_list'))

    self.assertEqual(match.func, views.product_list)

def test_product_detail_url_resolves_to_product_detail_view(self):
    match = resolve(reverse('product_detail', args=[self.product.pk]))

    self.assertEqual(match.func, views.product_detail)

def test_product_create_url_resolves_to_product_create_view(self):
    match = resolve(reverse('product_create'))

    self.assertEqual(match.func, views.product_create)

def test_product_update_url_resolves_to_product_update_view(self):
    match = resolve(reverse('product_update', args=[self.product.pk]))

    self.assertEqual(match.func, views.product_update)

def test_product_delete_url_resolves_to_product_delete_view(self):
    match = resolve(reverse('product_delete', args=[self.product.pk]))

    self.assertEqual(match.func, views.product_delete)

def test_shipment_routes_resolve_to_expected_views(self):
    self.assertEqual(resolve(reverse('shipment_list')).func, views.shipment_list)
    self.assertEqual(resolve(reverse('shipment_create')).func, views.shipment_create)
    self.assertEqual(resolve(reverse('shipment_detail', args=[self.shipment.pk]))).func,
views.shipment_detail)
    self.assertEqual(resolve(reverse('shipment_add_item', args=[self.shipment.pk]))).func,
views.shipment_add_item)
    self.assertEqual(
        resolve(reverse('shipment_delete_item', args=[self.shipment.pk,
self.shipment_item.pk]))).func,
views.shipment_delete_item,
    )

def test_about_url_resolves_to_about_view(self):
    match = resolve(reverse('about'))

    self.assertEqual(match.func, views.about)

def test_product_list_reverse_returns_catalog_root(self):
    self.assertEqual(reverse('product_list'), '/')

def test_product_list_get_returns_200(self):
    response = self.client.get(reverse('product_list'))

    self.assertEqual(response.status_code, 200)

def test_product_detail_get_returns_200(self):
    response = self.client.get(reverse('product_detail', args=[self.product.pk]))

    self.assertEqual(response.status_code, 200)

def test_product_detail_missing_product_returns_404(self):
    response = self.client.get(reverse('product_detail', args=[999]))

    self.assertEqual(response.status_code, 404)

```

```
def test_about_get_returns_200(self):  
    response = self.client.get(reverse('about'))  
  
    self.assertEqual(response.status_code, 200)
```

ВЫВОДЫ

В результате выполнения практической работы был расширен проект каталога аккумуляторов на Django. В приложении появились роли пользователей, пользовательские формы управления товарами, оптовое ценообразование и механизм формирования партии товара на отправку.

Товаровед получил возможность создавать, редактировать и удалять товары через интерфейс каталога. Менеджер продаж получил возможность создавать собственные партии, добавлять товары и задавать дополнительную скидку. Гость оставлен только с правами просмотра, а администратор имеет доступ ко всем операциям.

Расчет партии учитывает обычную цену, цену мелкого опта, цену крупного опта и дополнительную скидку на общую сумму. Для защиты данных реализовано ограничение: менеджер не может открыть или изменить партию другого менеджера.

Новый функционал покрыт автоматизированными тестами. Всего выполнено 46 тестов, все они прошли успешно. Покрытие forms.py составляет 100%, views.py - 92%, а общее покрытие проекта - 96%.

Таким образом, цель практической работы достигнута.